Kelompok 3

PEMINDAIAN DOKUMEN OTOMATIS

Anggota :
Mahda Annisa (2408107010036)
M. Rayan Zishan (2408107010063)
Zanna Zikraana (2408107010094)
Cut Farah Salsabila (2408107010100)

LATAR BELAKANG

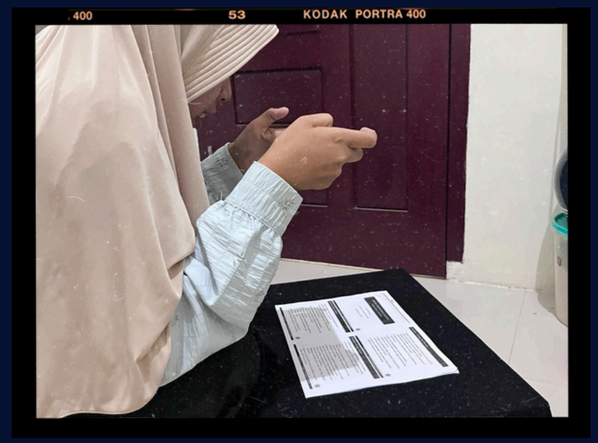
Permasalahan

- Foto dokumen menggunakan kamera ponsel sering menghasilkan bentuk kertas yang miring atau tidak beraturan.
- Bayangan dan pencahayaan tidak merata dapat mengganggu keterbacaan dokumen.
- Latar belakang yang menyerupai warna kertas menyulitkan pemisahan dokumen dari lingkungan sekitarnya.

Solusi yang dikembangkan

Program pemindai dokumen otomatis menggunakan Python dan OpenCV untuk mendeteksi sudut dokumen, meluruskan perspektif, dan menghasilkan citra hitam-putih agar tulisan lebih mudah dibaca

TUJUAN PROYEK



1. Mendeteksi empat sudut dokumen secara otomatis.
2. Meluruskan dokumen menggunakan transformasi perspektif.
3. Menghasilkan citra dengan rasio ukuran A4.
4. Mengubah dokumen menjadi citra biner hitam-putih.
5. Membandingkan metode thresholding Otsu, Adaptive Mean, dan Adaptive Gaussian.
6. Menguji sistem pada foto dengan kondisi yang berbeda.

INI CARA KAMI MENYIAPKAN DATASET!

KAMI PAKAI 5 JENIS DOKUMEN UNTUK MEMASTIKAN VARIASI KONTEN DAN KONTRAS:"

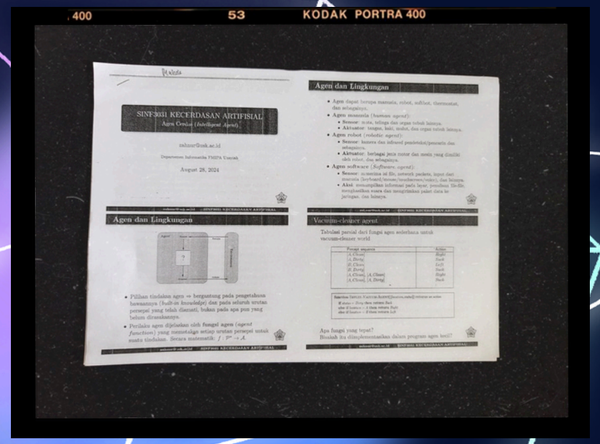
- **HALAMAN BUKU**
- **STRUK BELANJA**
- **KERTAS TUGAS TULISAN TANGAN**
- **BROSUR**
- **FOTOKOPI (HASIL FOTOKOPIAN, HITAM-PUTIH AGAK PUDAR)**

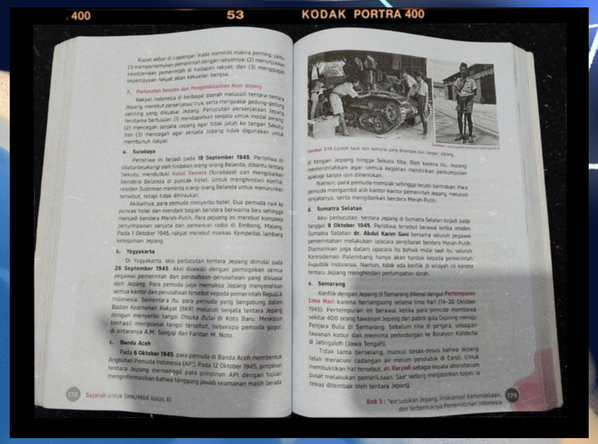
SETIAP FOTO DICATAT DI GROUND_TRUTH.CSV DENGAN 3 KOLOM:"
FILENAME — NAMA FILE
KATEGORI — IDEAL / CAHAYA / SUDUT / SULIT
SUDUT_TERLIHAT_PENUH — YA/TIDAK (APAKAH 4 SUDUT KERTAS TERLIHAT PENUH)

KOLOM TERAKHIR DIPAKAI NANTI SEBAGAI KUNCI JAWABAN UNTUK MENILAI APAKAH PROGRAM BERHASIL MENDETEKSI SUDUT ATAU TIDAK.

Kategori	Tujuan
Ideal (~15° miring)	seharusnya mudah dideteksi
Miring besar (30–45° + diputar)	Uji ketahanan warp perspektif
Ada bayangan tangan/HP	Uji ketahanan Canny & threshold
Sulit (latar mirip, terlipat, sudut	adu mekanik

DATASET



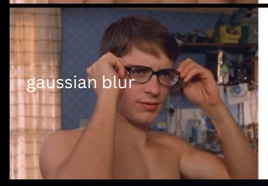


Kondisi Foto	Jumlah
Ideal	5
Pencahayaan atau bayangan	4
Sudut miring atau rotasi	12
kondisi sulit	8
Total	29

Jenis dokumen meliputi halaman buku, struk, tulisan tangan, fotokopi, dan brosur.

START

noise



lebih bersih

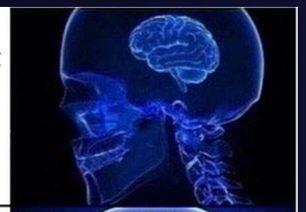
Dataset: 24 foto, 4 kategori, 5 dokumen, 1 CSV

Kelompok 3 bilek:

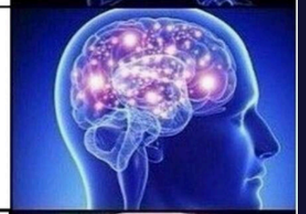
'tinggal deteksi 4 sudut, gampang kan?'

'tinggal deteksi 4 sudut, gampang kan?'

Memproses langsung foto berwarna dengan tekstur meja dan bayangan

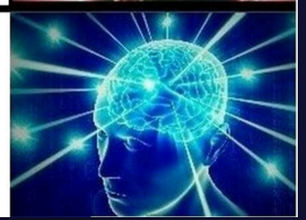


Ubah gambar ke Grayscale (cv2.cvtColor) biar komputasi lebih ringan



Tambahkan Gaussian Blur 5x5 untuk menghapus noise

Pakai algoritma Canny untuk menyisakan garis batas ekstrem saja seperti buku mewarnai!



Preprocessing

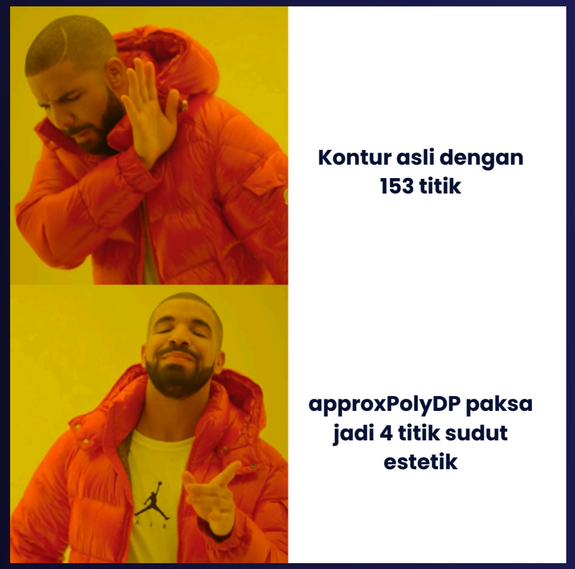
Kita mengambil foto asli yang berwarna, membuang warnanya menjadi abu-abu agar komputasi lebih ringan. Setelah itu, kita memburamkannya sedikit (Gaussian Blur) supaya bintik noise dari kamera atau tekstur meja hilang. Terakhir, kita mencari garis-garis kontras tinggi (Canny) yang memisahkan area terang (kertas) dan area gelap (latar belakang)

```python
def _preprocess_variant(image, variant="clahe_normal"):
    # ...
    # 1. Ubah ke Grayscale
    gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

    # 2. Kurangi noise dengan Gaussian Blur
    blurred = cv2.GaussianBlur(gray_eq, (5, 5), 0)

    # 3. Deteksi Tepi menggunakan Canny
    median = float(np.median(blurred))
    edges = cv2.Canny(blurred, low, high)
```

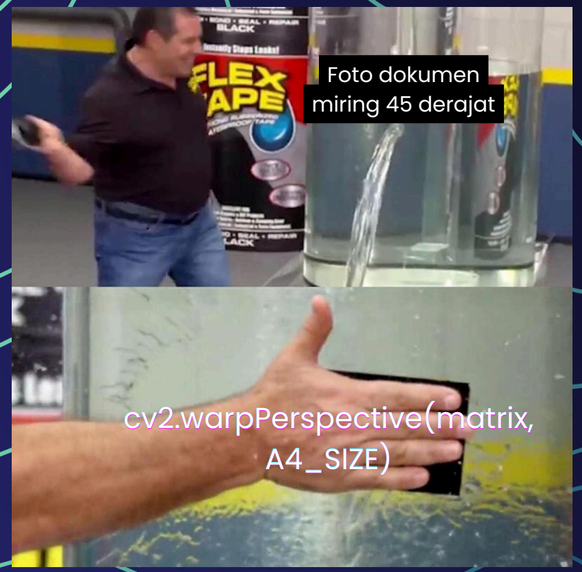
Mencari 4 Sudut

Dari sekian banyak garis putih hasil Canny di tahap sebelumnya, kita menelusuri garis mana saja yang menyambung penuh menjadi kurva tertutup (findContours). Karena tepi kertas di foto mungkin sedikit melengkung atau bergerigi, kita paksa luruskan dan sederhanakan bentuknya (approxPolyDP). Jika bentuk yang disederhanakan itu punya tepat 4 titik sudut, program akan menguncinya sebagai dokumen.

```python
def _kumpulkan_kandidat(edges_closed, image_area, ...):
    # 1. Cari semua kontur tertutup
    contours, _ = cv2.findContours(padded.copy(), mode, cv2.CHAIN_APPROX_SIMPLE)

    for contour in contours[:80]:
        perimeter = cv2.arcLength(contour, True)

        # 2. Sederhanakan kontur berdasarkan nilai epsilon
        for eps in epsilon_values:
            approx = cv2.approxPolyDP(contour, eps * perimeter, True)

            # 3. Filter: Cari yang punya tepat 4 titik
            if len(approx) == 4 and cv2.isContourConvex(approx):
                quads.append(approx)
```



```python
def perspective_warp(image, corners, width=840, height=1188):
    # 1. Koordinat 4 sudut miring di foto asli
    src = order_points(corners).astype(np.float32)

    # 2. Koordinat 4 sudut tujuan (tegak lurus A4)
    dst = np.array([[0, 0], [width - 1, 0], [width - 1, height - 1], [0, height

    # 3. Hitung rumus matriks & luruskan
    matrix = cv2.getPerspectiveTransform(src, dst)
    return cv2.warpPerspective(image, matrix, (width, height))
```



Meluruskan Dokumen

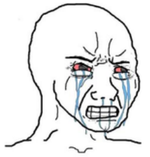
Dari 4 titik sebelumnya, kita pakai "kanvas kosong" berukuran A4 (840x1188 piksel) dan program akan menghitung secara matematika perubahannya (getPerspectiveTransform), lalu menarik paksa isi kertas miring tersebut dan menempelkannya pas di sudut-sudut kanvas A4 yang baru (warpPerspective).

Thresholding

Menerima gambar kertas yang sudah lurus, tapi pencahayaannya mungkin belang karena bayangan tangan/HP. Daripada memukul rata standar hitam-putih untuk satu gambar full (Metode Otsu) yang bisa bust area bayangan jadi hitam legam, kita memakai Metode Adaptive. Di sini, program memecah gambar ke blok-blok area kecil, mengecek cahayanya masing-masing, dan mengubah warnanya jadi hitam-putih secara independen. Hasilnya, teks di area gelap tetap terlihat tajam.

OTSU BE LIKE:

Bayangan pada gambar buat hasil crop kurang bisa dibaca



Reaksi Threshold (Otsu) :

```python
def make_threshold_comparison(warped, block_size=35, C=9, ...):
    # Otsu (Global) - Sering gagal karena bayangan HP
    _, otsu = cv2.threshold(smooth, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # Adaptive Gaussian - Hitung ambang batas per blok kecil
    adaptive_gaussian = cv2.adaptiveThreshold(
        smooth, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, block_size, C
    )
```

EKSPERIMEN NILAI EPSILON

Eksperimen dilakukan untuk mengetahui pengaruh nilai epsilon terhadap penyederhanaan kontur menggunakan approxPolyDP.

Nilai Epsilon	Efek pada Garis/Sudut Kertas (Fokus Pengamatan)
0,01	Terlalu detail: Toleransi sangat kecil. Garis pinggir masih bergerigi dan titik sudutnya terlalu banyak (belum menyederhana jadi 4 titik).
0,015	Transisi awal: Bentuk mulai dirapikan. Kita melihat seberapa banyak titik-titik yang tidak perlu mulai dibuang.
0,02	Titik ideal: Penyederhanaan yang pas. Garis lurus dengan baik dan biasanya berhasil mendeteksi tepat di 4 sudut kertas.
0,03	Mulai longgar: Toleransi agak besar. Kita mengecek apakah bentuk persegi kertasnya masih utuh atau mulai peyot/terdistorsi.
0,04	Terlalu cuek (Bahaya): Ada risiko tinggi sudut asli kertas malah ikut terpotong oleh program, sehingga bentuknya malah jadi segitiga atau tak beraturan.

Nilai epsilon yang kecil mempertahankan lebih banyak detail, sedangkan nilai yang lebih besar menyederhanakan kontur dengan toleransi lebih tinggi.

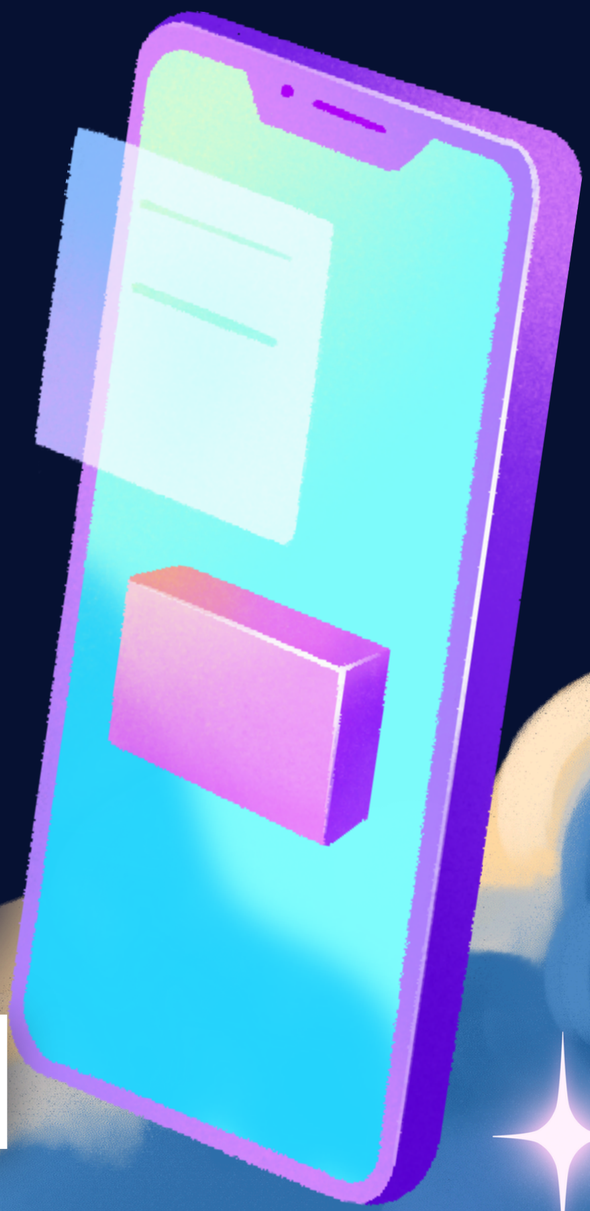
ANALISIS DAN KETERBATASAN

Hasil analisis

- Gaussian Blur membantu mengurangi noise sebelum deteksi tepi.
- Canny dan kontur membantu menemukan kandidat batas dokumen.
- Transformasi perspektif membuat ukuran keluaran lebih konsisten.
- Adaptive Gaussian memberikan hasil visual yang cukup baik pada contoh yang diamati.

Keterbatasan

- Program masih dapat memilih kontur yang bukan dokumen sebenarnya.
- Bayangan, lipatan, dan sudut tertutup dapat mengganggu deteksi.
- Ukuran A4 tidak menjamin batas dokumen telah terdeteksi dengan tepat.
- Thresholding dapat menghilangkan tulisan tipis atau menghasilkan bintik.
- Evaluasi belum mengukur akurasi koordinat empat sudut secara langsung.



KESIMPULAN DAN PENGEMBANGAN

Kesimpulan

Program pemindai dokumen otomatis berhasil dibangun menggunakan Python dan OpenCV. Pipeline mencakup deteksi tepi, pencarian kontur, transformasi perspektif, dan thresholding. Sebanyak 29 foto berhasil diproses menurut status program.

Saran pengembangan

1. Memperbaiki pemilihan kontur dan batas dokumen.
2. Menambahkan pemeriksaan visual hasil pelurusan.
3. Mengevaluasi koordinat sudut dengan data acuan.
4. Menguji parameter thresholding pada lebih banyak kondisi pencahayaan.

THANK YOU!

Jokowi: Kalau Semua Bertanya, Saya Tanya Siapa?